

• BROWSE AND AUTOMATE • TEST WEB APPS • CONTROL AND AUDIT AGENT ACCESS

A Security-First Browser for AI Agents.

- ▶ Browse, scrape, and automate faster. Pure Rust. No Chromium or V8.
 - ▶ Protect agents from the web. Network policies, sandboxing, and reviewable sessions.
 - ▶ Use agents to test the web. Capture, edit, replay, and compare HTTP traffic.
- ★ 600+ GitHub stars • Apache 2.0 • local-first, no hosted service

■ WHY NOW

Giving agents a browser changes both sides of web security.

TODAY

Browsers built for people

- ✗ Browsers designed for people are too slow and memory-intensive for agent tasks
- ✗ Malicious pages can manipulate agents into harmful actions, such as leaking credentials
- ✗ HTTP visibility and request manipulation are needed for red teaming

WITH H5I

A browser built for agents

- ✓ ~3× faster reads with ~86% less peak memory on simple websites
- ✓ Sandboxing and network rules limit what a manipulated agent can access
- ✓ Inspect, edit, replay, and compare traffic through the same browser

Security for AI and AI for Security both need a better browser.

SPEED

Too slow for routine agent tasks

Agents spend time waiting for pages during repeated reads and scraping.

ATTACK

Red teaming needs HTTP visibility

Request headers and bodies are needed to investigate vulnerabilities.

DEFENSE

Every page is untrusted input

Prompt injections can manipulate an agent into harmful actions.

SPEED

Memory limits parallel sessions

Heavy browsers reduce how many agent tasks can run on one machine.

ATTACK

Testing needs request manipulation

Agents need to edit and replay requests to verify an attack.

DEFENSE

Browser access carries authority

Logged-in sessions and internal services can be misused without access limits.

Protect agents from the web.

Use agents to test the web.

SPEED

~3× faster page reads

Compared with headless Chromium in our benchmarks on simple websites.

ATTACK

Inspect browser HTTP traffic

Capture requests and responses, including headers and bodies.

DEFENSE

Limit the damage of prompt injection

Mark page content as untrusted; use policies and isolation to limit its impact.

SPEED

~86% less peak memory

Compared with headless Chromium in the same simple-site benchmarks.

ATTACK

Edit, replay, and compare

Modify captured requests and verify responses without a separate MITM proxy or repeater.

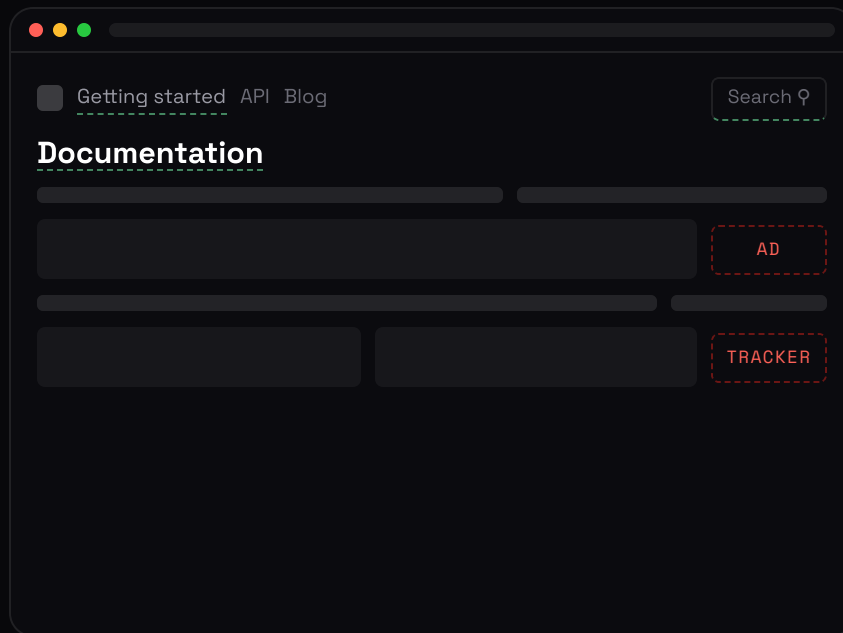
DEFENSE

Control and audit agent access

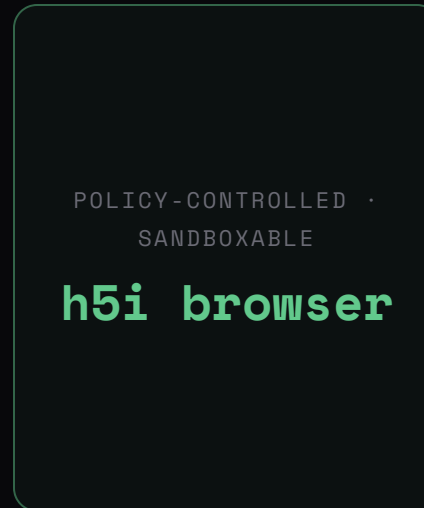
Restrict destinations, isolate credentials, and record allowed and denied requests.

Browse, scrape, and automate. Sandbox the workflow.

THE WEB · WHAT A PAGE LOADS

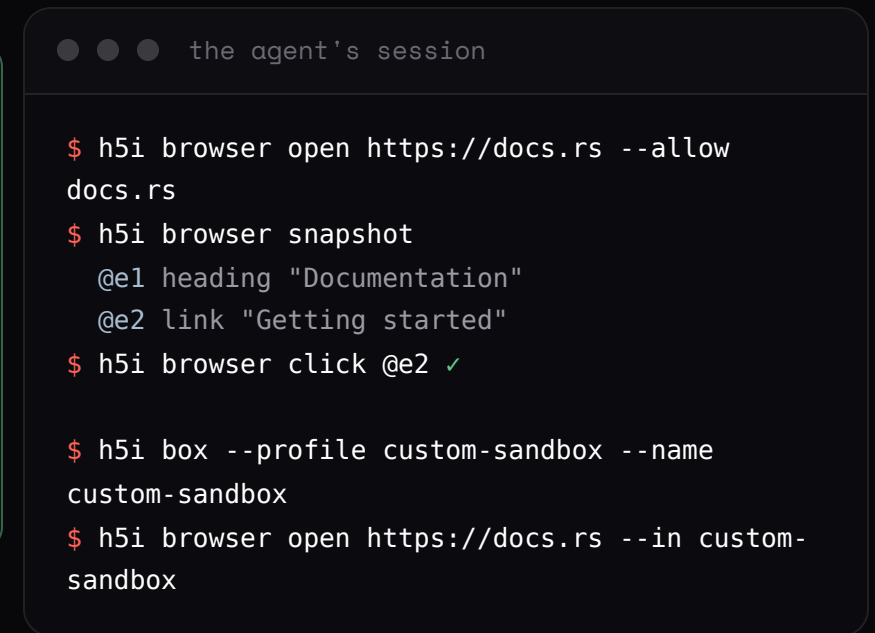


▶▶▶
PAGE BYTES



▶▶▶
OUTLINE, NOT PIXELS

THE AGENT · WHAT IT RUNS, AND WHAT COMES BACK



* Simple-site benchmarks vs headless Chromium · browser-only or full-workflow isolation

~3x

faster reads*

~86%

less peak memory*

Sandboxed

browser or agent + browser

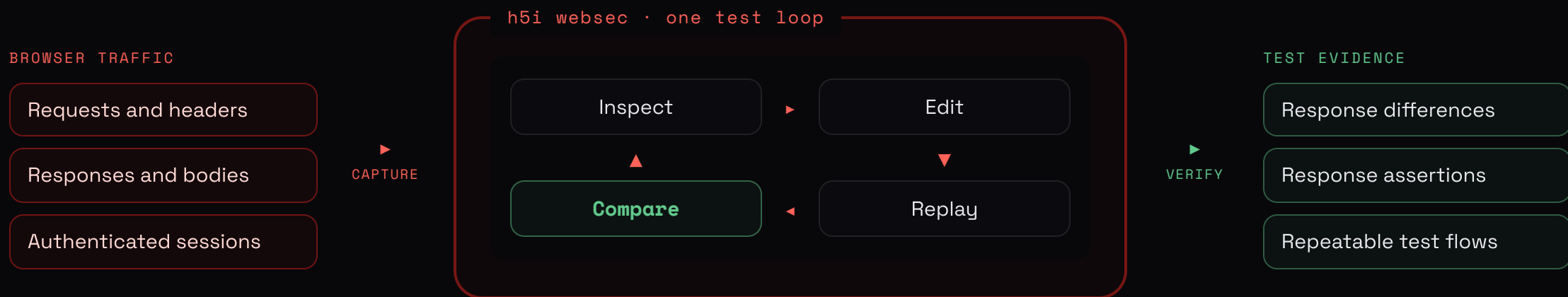
Auditable

every action is recorded

■ WEB SECURITY TESTING

Inspect and test web apps from **the same browser**.

The optional websec plugin lets agents capture, edit, replay, and compare browser traffic. No separate MITM proxy, CA setup, or repeater.



Browse the app. **Test its traffic.** Re-run the flow for **authorized security testing**.

■ TRACTION

Agent-heavy developers are
already **pulling** this.

600+

GitHub stars

60+

forks

20+

contributors

20+

releases since March

Start with developers already giving agents **browser access**.

Developers use agents to browse and test web apps. They need page automation, HTTP testing, and controlled access together.

BEACHHEAD · BUYS FIRST

Developers building browser-enabled agents

Need local browsing, structured page output, HTTP security tests, and reviewable records.

EXPANSION →

Agent platforms

Run many concurrent browser sessions with lower memory use.

Platform and DevEx teams

Turn repeatable web tests into checks across local development and CI.

Security teams

Use agents for CTFs, security research, and authorized pentesting.

Based on dozens of developer interviews during PhD research in AI and systems security.

■ WHY H5I WINS

Page automation. HTTP testing. One security-first browser.

Conventional browser automation

Broad website compatibility and mature automation. HTTP security testing adds a separate workflow.

Browser + proxy tools

Rich traffic inspection, with browser state, proxy setup, and test scripts to coordinate.

Sandboxing an existing browser

Constrains execution and access, but does not itself add HTTP inspection or replay.

h5i

A pure-Rust browser combining page automation, HTTP capture and replay, network policies, and configurable isolation.

Protect agents from the web. Use agents to test the web.

We are making h5i the **default browser** for AI agents.

Why now

- ▶ Agents are increasingly browsing websites and testing web apps with less human supervision.
- ▶ Agents need page automation, HTTP inspection, and access policies in one workflow.

Why us

- ▶ 500+ GitHub stars, 20+ contributors, and third-party tools built by users.
- ▶ Grounded in PhD research and dozens of developer interviews on agent security.

Next 12 weeks

- ▶ Expand website compatibility and repeatable HTTP security test flows.
- ▶ Convert 10 design partners and land the first paid pilots.

Protect agents from the web.

Use agents to test the web.

h5i.dev

github.com/h5i-dev/h5i